System Design

The process of designing the architecture of a software system that it works reliably at scale.

Focuses on :-
- How services talk to each other
- How data is stored, how requests are handled
- How failures are managed, etc.

Example :- If we build a hospital management system, and suddenly 50 branches start using it, we must design:
1) Load balancing
2) DB scaling
3) Caching
4) ~~Failure~~ / Failover systems.

Otherwise system will freeze during peak hours.

Backend Architecture :-
1) Client (Web / Mobile)
2) API Servers
3) Load balancer
5) Cache
4) Database
6) Message Queues.
7) Storage
8) Monitoring

Instead of one big server

We design:-
- Multiple servers
- Caching layers
- Failure handling
- Scalable databases
- Async processing

Real Use case

Instagram App
- Millions upload photos
- Need CDN for fast image delivery
- Need caching for feeds.
- Need database sharding

Payment systems
- Must ensure data consistency
- Cannot lose transactions
- Needs high availability

Trade-offs & limitations

You cannot optimize everything at once.

For ex

* • Adding caching improves speed
  • But it introduces cache invalidation problems.

* • Using microservices improves scalability.
  • But increases system complexity.

* • Using distributed systems improves fault tolerance.
  • But debugging becomes harder.

Types of System Design

1) High Level Design • Architecture Diagram • Data Flow
   • Components.

2) Low level design :- • DB schema • API structure
   • Class design • Data models.

* Suppose you are designing a URL shortener :-
  You must decide:
  → How to generate short IDs
  → Where to store mappings
  → How to handle millions of redirects
  → How to scale globally
  → How to avoid downtime.

These thinking process is system design.